

Grundlagen

SQL

Inhaltsverzeichnis

SQL - Grundlagen	4
Inhalte	4
Durchgängiges Beispiel	4
Was ist SQL?	5
Wichtige Sprachbereiche	5
Merke	5
Relationale Datenbanken	6
Beziehungen	6
Warum mehrere Tabellen?	6
Datentypen	7
Beispiel	7
Warum Datentypen wichtig sind	7
NULL	8
Prüfen auf NULL	8
NOT NULL	8
Merke	8
CREATE TABLE	9
Zweite Tabelle	9
Reihenfolge	9
ALTER TABLE	10
Achtung	10
DROP TABLE	11
Achtung	11
TRUNCATE TABLE	12
Unterschied zu DELETE	12
Besondere Rolle	12
Merke	12
INSERT - Daten anlegen	13
Mehrere Datensätze	13
CRUD	13
SELECT - Daten lesen	14
Empfehlung	14
CRUD	14
WHERE - Daten filtern	15
ORDER BY - Daten sortieren	16
GROUP BY - Daten gruppieren	17
HAVING	18
UPDATE - Daten ändern	19
Gefahr ohne WHERE	19
CRUD	19

DELETE - Daten löschen	20
CRUD	20
CRUD	21
Beispiel	21
PRIMARY KEY	22
Eigenschaften	22
UNIQUE Constraint	23
PRIMARY KEY und UNIQUE	23
Zusammengesetztes UNIQUE	23
NULL	23
Alternative Schlüssel	24
Natürliche und künstliche Schlüssel	25
Natürlicher Schlüssel	25
Künstlicher Schlüssel	25
Abwägung	25
UUID	26
Vorteile	26
Nachteile	26
Merke	26
Zusammengesetzte Schlüssel	27
Alternative	27
FOREIGN KEY	28
Zweck	28
Referenzielle Integrität	29
Löschen referenzierter Datensätze	29
NOT NULL, CHECK und DEFAULT	30
NOT NULL	30
CHECK	30
DEFAULT	30
Zweck	30
Normalisierung	31
Ausgangstabelle	31
Aufteilung	31
Vorteile	31
Normalformen	31
Denormalisierung	32
Beispiel	32
Vorteile	32
Nachteile	32
Merke	32
JOINS	33
INNER JOIN	33
LEFT JOIN	33
Weitere JOIN-Arten	33

Indizes	34
Idee	34
Vorteile	34
Nachteile	34
Merke	34
Abfrageoptimierung	35
Nur benötigte Spalten	35
Geeignete Filter	35
Indizes	35
Unnötige Joins vermeiden	35
Denormalisierung	35
Übungen	36
1 – Tabelle anlegen	36
2 – CRUD	36
3 – UNIQUE	36
4 – Fremdschlüssel	36
5 – Zusammengesetzter Schlüssel	36
6 – Normalisierung	36
7 – Denormalisierung	37
SQL Cheatsheet	38
Tabelle	38
Einfügen	38
Lesen	38
Filtern	38
Ändern	38
Löschen	38
Sortieren	38
Gruppieren	38
Join	38
Constraints	39
Quellen	40

SQL - Grundlagen

Dieser Bereich vermittelt relationale Datenbankkonzepte und SQL möglichst herstellerunabhängig. Produktspezifische Besonderheiten gehören nicht hierher, sondern in den Bereich Werkzeuge/.

Inhalte

- relationale Datenbanken
- Tabellen und Datentypen
- DDL und DML
- CRUD
- Schlüssel und Constraints
- Fremdschlüssel
- zusammengesetzte Schlüssel
- natürliche und künstliche Schlüssel
- UUID
- Normalisierung
- Denormalisierung
- Joins
- Indizes
- Abfrageoptimierung
- Übungen und Referenz

Durchgängiges Beispiel

Viele Kapitel verwenden eine kleine Schulverwaltung mit Tabellen wie:

- Schüler
- Klasse
- Lehrkraft
- Fach
- Kurs
- Teilnahme

Dadurch bleibt der fachliche Kontext über die Kapitel hinweg stabil.

Was ist SQL?

SQL steht für **Structured Query Language**.

SQL wird verwendet, um relationale Datenbanken zu definieren, Daten zu speichern, abzufragen, zu verändern und zu löschen.

Typische Aufgaben:

```
CREATE TABLE ...  
INSERT INTO ...  
SELECT ...  
UPDATE ...  
DELETE ...
```

SQL beschreibt in erster Linie **was** mit Daten geschehen soll. Das Datenbanksystem entscheidet intern, **wie** die Anweisung möglichst effizient ausgeführt wird.

Wichtige Sprachbereiche

DDL - Data Definition Language

Struktur der Datenbank:

```
CREATE TABLE  
ALTER TABLE  
DROP TABLE
```

DML - Data Manipulation Language

Daten verändern:

```
INSERT  
UPDATE  
DELETE
```

DQL - Data Query Language

Daten lesen:

```
SELECT
```

Merke

SQL ist eine Sprache zur Arbeit mit relationalen Datenbanken. Konkrete Datenbanksysteme können zusätzliche Funktionen oder abweichende Details besitzen.

Relationale Datenbanken

Eine relationale Datenbank speichert Daten in Tabellen.

Beispiel:

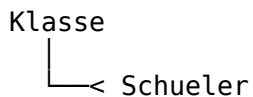
SchuelerID	Name	Klasse
1	Mia	9a
2	Tim	9a

Eine Zeile beschreibt einen Datensatz.

Eine Spalte beschreibt ein Attribut.

Beziehungen

Tabellen können miteinander verbunden werden.



Eine Klasse kann viele Schülerinnen und Schüler enthalten.

Warum mehrere Tabellen?

Mehrere Tabellen helfen,

- Wiederholungen zu vermeiden,
- Daten konsistent zu halten,
- Beziehungen sichtbar zu machen,
- Änderungen einfacher durchzuführen.

Datentypen

Spalten besitzen Datentypen.

Häufige SQL-Datentypen sind:

Typ	Verwendung
INTEGER	ganze Zahlen
DECIMAL	genaue Dezimalzahlen
VARCHAR	Texte begrenzter Länge
TEXT	längere Texte
DATE	Datum
TIMESTAMP	Datum und Uhrzeit
BOOLEAN	Wahr/Falsch

Beispiel

```
CREATE TABLE Schueler (  
  SchuelerID INTEGER,  
  Name VARCHAR(100),  
  Geburtsdatum DATE  
);
```

Warum Datentypen wichtig sind

Datentypen helfen bei:

- Validierung,
- Speicherbedarf,
- Sortierung,
- Berechnungen,
- Datenqualität.

NULL

NULL bedeutet:

Kein Wert ist vorhanden.

NULL ist nicht dasselbe wie:

0

""

false

Prüfen auf NULL

```
SELECT *  
FROM Schueler  
WHERE Telefonnummer IS NULL;
```

Nicht:

```
WHERE Telefonnummer = NULL
```

NOT NULL

Wenn ein Wert zwingend vorhanden sein muss:

Name VARCHAR(100) NOT NULL

Merke

NULL bedeutet fehlender oder unbekannter Wert.

CREATE TABLE

Mit CREATE TABLE wird eine neue Tabelle angelegt.

```
CREATE TABLE Klasse (  
    KlasseID INTEGER PRIMARY KEY,  
    Bezeichnung VARCHAR(10) NOT NULL  
);
```

Zweite Tabelle

```
CREATE TABLE Schueler (  
    SchuelerID INTEGER PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    KlasseID INTEGER,  
    FOREIGN KEY (KlasseID)  
        REFERENCES Klasse(KlasseID)  
);
```

Reihenfolge

Wenn ein Fremdschlüssel auf eine andere Tabelle verweist, sollte die referenzierte Tabelle bereits existieren.

ALTER TABLE

Mit ALTER TABLE wird die Struktur einer bestehenden Tabelle verändert.

Typische Aufgaben:

```
ALTER TABLE Schueler  
ADD Email VARCHAR(200);
```

Je nach Datenbanksystem sind außerdem möglich:

```
ALTER TABLE ...  
DROP COLUMN ...
```

oder

```
ALTER TABLE ...  
RENAME COLUMN ...
```

Achtung

Die genaue Unterstützung einzelner ALTER TABLE-Varianten unterscheidet sich zwischen Datenbanksystemen.

DROP TABLE

Mit DROP TABLE wird eine Tabelle einschließlich ihrer Struktur entfernt.

DROP TABLE Schueler;

Achtung

Dabei gehen normalerweise auch die enthaltenen Daten verloren.

DROP TABLE entfernt die Tabelle selbst.

Das unterscheidet den Befehl von DELETE und TRUNCATE.

TRUNCATE TABLE

TRUNCATE TABLE entfernt alle Datensätze einer Tabelle, lässt die Tabellenstruktur jedoch bestehen.

TRUNCATE TABLE Schueler;

Unterschied zu DELETE

DELETE FROM Schueler;

und

TRUNCATE TABLE Schueler;

können beide alle Datensätze entfernen, besitzen aber unterschiedliche Eigenschaften.

Typischerweise gilt:

- DELETE kann mit WHERE einzelne Datensätze löschen.
- TRUNCATE löscht immer den gesamten Tabelleninhalt.
- TRUNCATE ist in vielen Systemen für das vollständige Leeren einer Tabelle optimiert.
- Details zu Transaktionen, Triggern und Identitätsspalten sind systemabhängig.

Besondere Rolle

TRUNCATE TABLE ist kein gewöhnlicher CRUD-Befehl für einzelne Datensätze.

Er ist ein administrativer bzw. struktureller Massenbefehl und sollte entsprechend vorsichtig eingesetzt werden.

Merke

DELETE löscht Daten gezielt. TRUNCATE leert eine Tabelle vollständig.

INSERT - Daten anlegen

Mit INSERT werden neue Datensätze gespeichert.

```
INSERT INTO Klasse (KlasseID, Bezeichnung)
VALUES (1, '9a');
```

```
INSERT INTO Schueler (SchuelerID, Name, KlasseID)
VALUES (1001, 'Mia', 1);
```

Mehrere Datensätze

```
INSERT INTO Schueler (SchuelerID, Name, KlasseID)
VALUES
    (1002, 'Tim', 1),
    (1003, 'Lea', 1);
```

CRUD

INSERT entspricht dem **Create** in CRUD.

SELECT - Daten lesen

Mit SELECT werden Daten abgefragt.

```
SELECT Name  
FROM Schueler;
```

Mehrere Spalten:

```
SELECT SchuelerID, Name  
FROM Schueler;
```

Alle Spalten:

```
SELECT *  
FROM Schueler;
```

Empfehlung

Für produktive Abfragen ist es häufig besser, benötigte Spalten ausdrücklich anzugeben.

CRUD

SELECT entspricht dem **Read** in CRUD.

WHERE - Daten filtern

Mit WHERE werden Datensätze anhand einer Bedingung ausgewählt.

```
SELECT *  
FROM Schueler  
WHERE KlasseID = 1;
```

Vergleichsoperatoren:

```
=  
<>  
<  
>  
<=  
>=
```

Mehrere Bedingungen:

```
SELECT *  
FROM Schueler  
WHERE KlasseID = 1  
      AND Name = 'Mia';
```

ORDER BY - Daten sortieren

Aufsteigend:

```
SELECT Name  
FROM Schueler  
ORDER BY Name ASC;
```

Absteigend:

```
SELECT Name  
FROM Schueler  
ORDER BY Name DESC;
```

Mehrere Kriterien:

```
SELECT *  
FROM Schueler  
ORDER BY KlasseID ASC, Name ASC;
```

GROUP BY - Daten gruppieren

GROUP BY wird häufig mit Aggregatfunktionen verwendet.

```
SELECT KlasseID, COUNT(*) AS Anzahl  
FROM Schueler  
GROUP BY KlasseID;
```

Typische Aggregatfunktionen:

- COUNT
- SUM
- AVG
- MIN
- MAX

HAVING

WHERE filtert einzelne Datensätze vor der Gruppierung.

HAVING filtert Gruppen nach der Gruppierung.

```
SELECT KlasseID, COUNT(*) AS Anzahl  
FROM Schueler  
GROUP BY KlasseID  
HAVING COUNT(*) > 20;
```

UPDATE - Daten ändern

```
UPDATE Schueler  
SET Name = 'Mia Müller'  
WHERE SchuelerID = 1001;
```

Gefahr ohne WHERE

```
UPDATE Schueler  
SET KlasseID = 2;
```

ändert **alle** Datensätze.

CRUD

UPDATE entspricht dem **Update** in CRUD.

DELETE - Daten löschen

```
DELETE FROM Schueler  
WHERE SchuelerID = 1001;
```

Ohne WHERE:

```
DELETE FROM Schueler;
```

werden alle Datensätze gelöscht.

Die Tabelle bleibt jedoch bestehen.

CRUD

DELETE entspricht dem **Delete** in CRUD.

CRUD

CRUD fasst vier grundlegende Datenoperationen zusammen.

CRUD	SQL
Create	INSERT
Read	SELECT
Update	UPDATE
Delete	DELETE

Beispiel

```
INSERT INTO Schueler (SchuelerID, Name)
VALUES (1, 'Mia');
```

```
SELECT *
FROM Schueler
WHERE SchuelerID = 1;
```

```
UPDATE Schueler
SET Name = 'Mia Müller'
WHERE SchuelerID = 1;
```

```
DELETE FROM Schueler
WHERE SchuelerID = 1;
```

CREATE TABLE gehört trotz des Wortes CREATE nicht zum CRUD-Create. CRUD-Create meint das Anlegen eines Datensatzes.

PRIMARY KEY

Ein Primärschlüssel identifiziert jeden Datensatz eindeutig.

```
CREATE TABLE Schueler (  
    SchuelerID INTEGER PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL  
);
```

Eigenschaften

Ein Primärschlüssel:

- ist eindeutig,
- darf nicht fehlen,
- sollte möglichst stabil bleiben.

Eine Tabelle besitzt genau einen Primärschlüssel. Dieser kann aus einer oder mehreren Spalten bestehen.

UNIQUE Constraint

UNIQUE stellt sicher, dass ein Wert oder eine Kombination von Werten nicht doppelt vorkommt.

```
CREATE TABLE Benutzer (  
    BenutzerID INTEGER PRIMARY KEY,  
    Benutzername VARCHAR(100) UNIQUE,  
    Email VARCHAR(200) UNIQUE  
);
```

PRIMARY KEY und UNIQUE

PRIMARY KEY	UNIQUE
identifiziert Datensatz genau einer pro Tabelle kann zusammengesetzt sein	erzwingt Eindeutigkeit mehrere möglich kann ebenfalls zusammengesetzt sein

Zusammengesetztes UNIQUE

UNIQUE (Vorname, Nachname, Geburtsdatum)

NULL

Das Verhalten von NULL in UNIQUE-Constraints kann sich zwischen Datenbanksystemen unterscheiden.

Alternative Schlüssel

Manchmal existieren mehrere Attribute, die einen Datensatz eindeutig identifizieren könnten.

Beispiel:

SchuelerID

Schuelernummer

Beide könnten eindeutig sein.

Einer wird als Primärschlüssel gewählt.

Die übrigen geeigneten Kandidaten werden als alternative Schlüssel betrachtet.

In SQL werden alternative Schlüssel häufig mit UNIQUE abgesichert.

Schuelernummer **VARCHAR(20) UNIQUE**

Natürliche und künstliche Schlüssel

Natürlicher Schlüssel

Besitzt eine fachliche Bedeutung.

Beispiele:

- ISBN
- Fahrzeug-Identifikationsnummer
- Personalnummer

Künstlicher Schlüssel

Wird ausschließlich zur Identifikation erzeugt.

Beispiel:

SchuelerID = 1001

Abwägung

Natürliche Schlüssel können verständlicher sein.

Künstliche Schlüssel sind häufig stabiler und einfacher für Beziehungen verwendbar.

UUID

Eine UUID ist eine sehr große Kennung.

Beispiel:

550e8400-e29b-41d4-a716-446655440000

UUIDs werden häufig als künstliche Schlüssel verwendet.

Vorteile

- können dezentral erzeugt werden,
- sehr geringe Kollisionswahrscheinlichkeit,
- gut für verteilte Systeme.

Nachteile

- schlechter lesbar,
- größer als einfache Integer-Schlüssel,
- können bei Indizes mehr Speicher benötigen.

Merke

UUIDs sind eine mögliche Form künstlicher Schlüssel, aber nicht automatisch die beste Wahl für jede Tabelle.

Zusammengesetzte Schlüssel

Ein Schlüssel kann aus mehreren Spalten bestehen.

Beispiel einer Zuordnungstabelle:

```
CREATE TABLE Teilnahme (  
    SchuelerID INTEGER,  
    KursID INTEGER,  
    PRIMARY KEY (SchuelerID, KursID)  
);
```

Die Kombination ist eindeutig.

Ein Schüler kann nicht zweimal demselben Kurs zugeordnet werden.

Alternative

Man könnte zusätzlich eine künstliche TeilnahmeID verwenden und die Kombination absichern:

```
UNIQUE (SchuelerID, KursID)
```

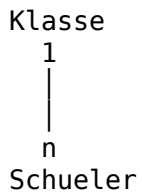
Welche Variante sinnvoller ist, hängt vom Datenmodell ab.

FOREIGN KEY

Ein Fremdschlüssel verweist auf einen Schlüssel einer anderen Tabelle.

```
CREATE TABLE Schueler (  
  SchuelerID INTEGER PRIMARY KEY,  
  Name VARCHAR(100),  
  KlasseID INTEGER,  
  FOREIGN KEY (KlasseID)  
    REFERENCES Klasse(KlasseID)  
);
```

Dadurch entsteht eine Beziehung:



Zweck

Fremdschlüssel helfen, ungültige Verweise zu verhindern.

Referenzielle Integrität

Referenzielle Integrität bedeutet:

Ein Fremdschlüssel darf nicht auf einen nicht vorhandenen Datensatz verweisen.

Beispiel:

Wenn `Schueeler.KlasseID = 5` gespeichert wird, sollte die Klasse mit `KlasseID = 5` existieren.

Löschen referenzierter Datensätze

Mögliche Strategien sind:

- RESTRICT
- CASCADE
- SET NULL

Beispiel:

```
FOREIGN KEY (KlasseID)
REFERENCES Klasse(KlasseID)
ON DELETE SET NULL
```

Die genaue Unterstützung und Voreinstellung hängt vom Datenbanksystem ab.

NOT NULL, CHECK und DEFAULT

NOT NULL

Name `VARCHAR(100) NOT NULL`

Wert muss vorhanden sein.

CHECK

`CHECK (Punkte >= 0)`

Wert muss eine Bedingung erfüllen.

DEFAULT

Status `VARCHAR(20) DEFAULT 'aktiv'`

Ohne expliziten Wert wird der Standardwert verwendet.

Zweck

Constraints verlagern wichtige Regeln direkt in die Datenbank und verbessern die Datenqualität.

Normalisierung

Normalisierung reduziert unnötige Datenwiederholungen.

Ausgangstabelle

Schueler	Klasse	Klassenlehrer
Mia	9a	Frau Müller
Tim	9a	Frau Müller
Lea	9a	Frau Müller

9a und Frau Müller werden mehrfach gespeichert.

Aufteilung

Klasse

- KlasseID
- Bezeichnung
- Klassenlehrer

Schueler

- SchuelerID
- Name
- KlasseID

Vorteile

- weniger Redundanz,
- weniger Änderungsfehler,
- klarere Beziehungen.

Normalformen

Als grundlegende Orientierung:

1NF

Werte sind atomar und Tabellenstrukturen eindeutig.

2NF

Nicht-Schlüsselattribute hängen vom gesamten Schlüssel ab.

3NF

Nicht-Schlüsselattribute hängen nicht voneinander ab.

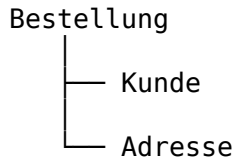
Für den Einstieg ist wichtiger, die zugrunde liegenden Redundanzprobleme zu verstehen als Definitionen auswendig zu lernen.

Denormalisierung

Denormalisierung bedeutet, Daten bewusst mehrfach oder weniger stark getrennt zu speichern. Das kann sinnvoll sein, wenn Lesezugriffe besonders schnell sein müssen.

Beispiel

Eine normalisierte Bestellung könnte Daten über mehrere Tabellen beziehen:



Für sehr häufig gelesene Berichte könnten ausgewählte Werte zusätzlich direkt in einer Auswertungstabelle gespeichert werden.

Vorteile

- weniger Joins,
- schnellere Lesezugriffe möglich,
- vereinfachte Reports.

Nachteile

- Daten werden doppelt gespeichert,
- höherer Speicherbedarf,
- Änderungen müssen synchron gehalten werden,
- Risiko inkonsistenter Daten.

Merke

Normalisierung verbessert Konsistenz. Denormalisierung kann gezielt Performance verbessern. Beides ist eine bewusste Architekturentscheidung.

JOINS

JOINS verbinden Datensätze aus mehreren Tabellen.

INNER JOIN

```
SELECT Schueler.Name, Klasse.Bezeichnung
FROM Schueler
INNER JOIN Klasse
    ON Schueler.KlasseID = Klasse.KlasseID;
```

Es erscheinen nur passende Datensätze.

LEFT JOIN

```
SELECT Schueler.Name, Klasse.Bezeichnung
FROM Schueler
LEFT JOIN Klasse
    ON Schueler.KlasseID = Klasse.KlasseID;
```

Alle Schülerinnen und Schüler erscheinen, auch wenn keine passende Klasse vorhanden ist.

Weitere JOIN-Arten

Der SQL-Standard kennt weitere Varianten. Die konkrete Unterstützung kann je Datenbanksystem variieren.

Indizes

Ein Index beschleunigt das Finden von Datensätzen.

```
CREATE INDEX idx_schueler_name  
ON Schueler(Name);
```

Idee

Ohne geeigneten Index muss ein Datenbanksystem möglicherweise viele oder alle Zeilen durchsuchen.

Ein Index erzeugt eine zusätzliche Suchstruktur.

Vorteile

- schnellere Suchabfragen,
- schnellere Joins in geeigneten Fällen.

Nachteile

- zusätzlicher Speicher,
- INSERT, UPDATE und DELETE können aufwendiger werden.

Merke

Ein Index beschleunigt Lesen nicht kostenlos. Er verursacht Speicher- und Pflegeaufwand.

Abfrageoptimierung

Einige grundlegende Regeln:

Nur benötigte Spalten

Statt:

```
SELECT *  
FROM Schueler;
```

wenn nur der Name benötigt wird:

```
SELECT Name  
FROM Schueler;
```

Geeignete Filter

```
WHERE SchuelerID = 1001
```

reduziert die zu verarbeitende Datenmenge.

Indizes

Häufig gesuchte oder für Joins verwendete Spalten können geeignete Indexkandidaten sein.

Unnötige Joins vermeiden

Nur Tabellen verbinden, deren Daten tatsächlich benötigt werden.

Denormalisierung

Bei speziellen Lese- oder Reportinganforderungen kann bewusst redundante Speicherung sinnvoll sein.

Optimierung sollte auf einem tatsächlichen Problem beruhen, nicht nur auf Vermutungen.

Übungen

1 - Tabelle anlegen

Erstelle eine Tabelle Fach mit:

- FachID
- Bezeichnung

FachID soll Primärschlüssel sein.

2 - CRUD

Führe nacheinander aus:

1. Datensatz anlegen
 2. Datensatz lesen
 3. Datensatz ändern
 4. Datensatz löschen
-

3 - UNIQUE

Erweitere eine Tabelle Benutzer so, dass Email eindeutig sein muss.

4 - Fremdschlüssel

Modelliere:

Klasse 1 --- n Schueler
mit SQL.

5 - Zusammengesetzter Schlüssel

Erstelle:

Teilnahme

- SchuelerID
- KursID

Die Kombination soll eindeutig sein.

6 - Normalisierung

Gegeben:

Schüler	Klasse	Klassenlehrer
---------	--------	---------------

Entwickle ein Modell mit zwei Tabellen.

7 - Denormalisierung

Nenne ein Beispiel, bei dem redundante Speicherung für eine häufig gelesene Auswertung sinnvoll sein könnte.

Begründe Vorteile und Risiken.

SQL Cheatsheet

Tabelle

```
CREATE TABLE Tabelle (  
    ID INTEGER PRIMARY KEY  
);
```

Einfügen

```
INSERT INTO Tabelle (ID)  
VALUES (1);
```

Lesen

```
SELECT *  
FROM Tabelle;
```

Filtern

```
SELECT *  
FROM Tabelle  
WHERE ID = 1;
```

Ändern

```
UPDATE Tabelle  
SET ...  
WHERE ...;
```

Löschen

```
DELETE FROM Tabelle  
WHERE ...;
```

Sortieren

```
ORDER BY Spalte ASC;
```

Gruppieren

```
GROUP BY Spalte;
```

Join

```
SELECT ...  
FROM A  
JOIN B ON A.ID = B.A_ID;
```

Constraints

PRIMARY KEY
FOREIGN KEY
UNIQUE
NOT NULL
CHECK
DEFAULT

Quellen

Für die Weiterentwicklung dieses Grundlagenbereichs sollten bevorzugt verwendet werden:

- ISO/IEC 9075 – SQL
- Dokumentationen etablierter relationaler Datenbanksysteme zur Gegenprüfung standardnaher Syntax
- Fachliteratur zu relationalen Datenbanken und Datenmodellierung

Produktspezifische Besonderheiten werden nicht in diesem Bereich dokumentiert, sondern unter Werkzeuge/.